

*Examen Final (CF)***ESIN : Semestre 6**

Matière Web Development I
Enseignant Saad NOUFEL
Date Mai 2026
Durée 2h00

uirblue !15 Course Outcomes	Exercice	Pourcentage
CO2 : Mastery of Basic and OOP principles in PHP to develop robust and scalable applications	3	45%
CO3 : Database access with PDO and web authentication via Sessions & Cookies	1 & 2	55%

Nom

Prénom

Numéro d'étudiant

Consignes :

- Répondez directement sur le sujet
- Vous disposez d'un cheatsheet en dernière page
- **Bonne chance !**

Exercice 1 Sessions & Cookies**(5 pts)**

On souhaite développer un système d'authentification pour **CampusMedia**, une vidéothèque universitaire en ligne.

Soit le formulaire HTML de connexion suivant :

```
1 <form method="POST" action="connexion.php">
2   <label>Login :</label>
3   <input type="text" name="login" required>
4
5   <label>Mot de passe :</label>
6   <input type="password" name="mot_de_passe" required>
7
8   <button type="submit">Se connecter</button>
9 </form>
```

Question 1 (2 pts) : Écrivez le code PHP du fichier `connexion.php` qui :

- Démarre la session
- Récupère le login et le mot de passe envoyés par le formulaire
- Vérifie que les deux champs ne sont pas vides, sinon affiche "Veuillez remplir tous les champs."

- Interroge la table `utilisateurs` avec PDO pour trouver l'utilisateur dont le `login` correspond
- Vérifie le mot de passe avec `password_verify()`
- En cas de succès : stocke le login dans `$_SESSION['utilisateur']` et le rôle dans `$_SESSION['role']`, crée un cookie `derniere_connexion` avec la date et l'heure actuelles (valide **30 jours**), redirige vers `catalogue.php`
- En cas d'échec : affiche "`Identifiants incorrects.`"

Question 2 (1.5 pts) : Écrivez le code PHP du **début** du fichier `catalogue.php` qui :

- Vérifie que l'utilisateur est connecté, sinon redirige vers `connexion.php`
- Affiche "`Bienvenue, [login] ! (rôle : [role])`" en récupérant les données depuis la session
- Si le cookie `derniere_connexion` existe, affiche "`Dernière connexion : [date]`", sinon "`Première connexion`"

Question 3 (1.5 pts) : Écrivez le code complet du fichier `deconnexion.php` qui :

- Démarre la session, supprime toutes les variables de session et détruit la session côté serveur
- Supprime le cookie `derniere_connexion`
- Redirige vers `connexion.php`

Exercice 2 PHP et PDO**(6 pts)**

On souhaite gérer le catalogue de films de **CampusMedia**. Les films sont stockés dans la table **FILMS** de la base de données **campusmedia_bd**.

Structure de la table **FILMS** :

Colonne	Type
id	INT AUTO_INCREMENT PRIMARY KEY
titre	VARCHAR(100)
realisateur	VARCHAR(80)
genre	VARCHAR(30)
annee	YEAR
note	DECIMAL(3,1)

Informations de connexion : `mysql:host=localhost;dbname=campusmedia_bd`, utilisateur : 'mediauser', mot de passe : 'media2026'

Question 1 (1 pt) : Écrivez la fonction `getConnexion()` : PDO permettant de se connecter à la base de données avec le mode d'erreur `ERRMODE_EXCEPTION` et le mode de fetch `FETCH_ASSOC` par défaut.

Question 2 (2 pts) : Écrivez la méthode statique `getAll(PDO $pdo, int $limit = 0): array` qui :

- Récupère tous les films depuis la base de données, triés par note décroissante
- Si `$limit > 0`, limite le nombre de résultats retournés
- Retourne un tableau d'objets `Film`

Question 3 (1.5 pts) : Écrivez la méthode `save(PDO $pdo): bool` qui insère le film courant (`$this`) dans la base de données en utilisant des **paramètres nommés** (`:label`). En cas de succès, mettez à jour `$this->id` avec l'identifiant généré.

Question 4 (1.5 pts) : Écrivez la méthode statique `findByGenre(PDO $pdo, string $genre): array` qui retourne tous les films d'un genre donné triés par note décroissante. Expliquez ensuite en une phrase pourquoi votre implémentation protège contre les injections SQL.

Explication :
.....

Exercice 3 POO en PHP**(9 pts)**

On souhaite modéliser les ressources disponibles dans la vidéothèque **CampusMedia**.

Question 1 (2 pts) : Créez une classe **abstraite** `Ressource` avec :

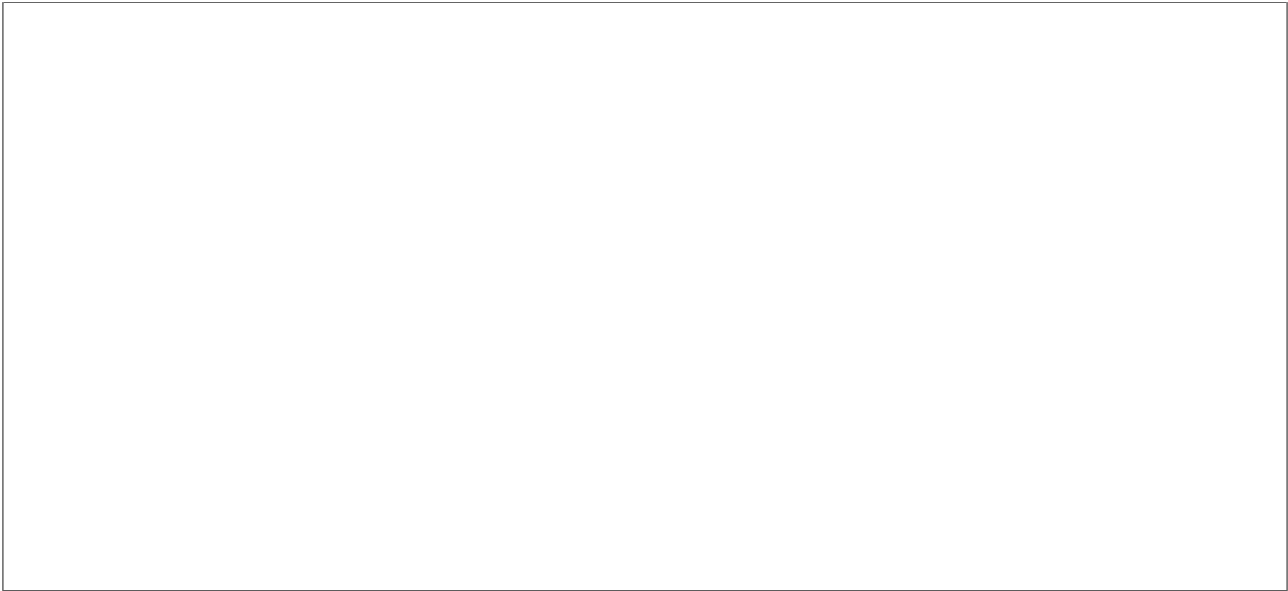
- Attributs **protégés** : `id` (int), `titre` (string), `dateAjout` (string)
- Un constructeur pour initialiser ces trois attributs
- Une méthode **abstraite** `getType(): string`
- Une méthode **concrète** `__toString()` qui retourne une chaîne contenant l'id, le titre et la date d'ajout

Question 2 (1 pt) : Créez une **interface** `Emprutable` avec les méthodes suivantes :

- `emprunter(string $nomEtudiant): bool` — retourne `true` si l'emprunt est réussi, `false` sinon
- `retourner(): bool` — retourne `true` si le retour est réussi, `false` sinon
- `getEmprunteur(): ?string` — retourne le nom de l'emprunteur ou `null` si la ressource est disponible

Question 3 (6 pts) : Créez une classe `Film` qui **hérite** de `Ressource` ET **implémente** `Emprutable` avec :

- Attributs propres : `realisateur` (string), `genre` (string), `dureeMinutes` (int), `estDisponible` (bool), `emprunteur` (?string)
- Un constructeur qui appelle `parent::__construct()` et initialise les attributs propres (`estDisponible = true` et `emprunteur = null` par défaut)
- Un **getter** pour `realisateur`
- Un **setter** pour `genre`
- Implémentation de `getType()` : retourne la chaîne "Film"
- Implémentation des **trois méthodes** de `Emprutable` :
 - `emprunter()` : réussi uniquement si le film est disponible
 - `retourner()` : réussi uniquement si le film est actuellement emprunté
 - `getEmprunteur()` : retourne l'emprunteur ou `null`
- Une méthode `__toString()` qui surcharge celle du parent pour inclure également le réalisateur, le genre, la durée et la disponibilité



QCM — Cochez la ou les bonnes réponses (5 pts)

Question 1 (1 pt) : Laquelle des affirmations suivantes est correcte concernant `session_start()` ?

- ☐ Elle peut être appelée après l'envoi de contenu HTML
- ☐ Elle doit être appelée avant tout output, en tout premier
- ☐ Elle crée automatiquement un cookie côté client avec l'ID de session
- ☐ Elle stocke les données de session dans le navigateur

Question 2 (1 pt) : Comment supprimer définitivement un cookie nommé `token` en PHP ?

- ☐ `unset($_COOKIE['token'])`
- ☐ `setcookie('token', "", time() - 3600, '/')`
- ☐ `deletetecookie('token')`
- ☐ `session_destroy()`

Question 3 (1 pt) : Parmi les instructions PDO suivantes, laquelle protège contre les injections SQL ?

- ☐ `$pdo->query("SELECT * FROM films WHERE genre = '$genre'");`
- ☐ `$stmt = $pdo->prepare("SELECT * FROM films WHERE genre = ?"); $stmt->execute([$genre]);`
- ☐ `$pdo->exec("SELECT * FROM films WHERE genre = " . $genre);`
- ☐ `$pdo->query("SELECT * FROM films WHERE genre = " . htmlspecialchars($genre));`

Question 4 (1 pt) : Lesquelles des affirmations suivantes sur les classes abstraites et les interfaces sont **vraies** ?

- ☐ Une classe abstraite peut contenir des méthodes avec implémentation
- ☐ Une interface peut contenir des méthodes avec implémentation (corps)
- ☐ Une classe peut hériter de plusieurs classes abstraites à la fois en PHP
- ☐ Une classe peut implémenter plusieurs interfaces à la fois en PHP

Question 5 (1 pt) : Que retourne `$stmt->fetch(PDO::FETCH_ASSOC)` quand la requête ne trouve aucun résultat ?

- ☐ `[]`
- ☐ `null`
- ☐ `false`
- ☐ Une exception `PDOException`

— Fin du sujet —

Web Development I — Cheatsheet

Chapitres 3, 4 & 5 — ESIN Ing3

-2pt CHAPITRE 3 : POO EN PHP

Classe abstraite

```
abstract class Ressource {
    protected int $id;
    public function __construct(int $id) {
        $this->id = $id;
    }
    abstract public function getType(): string;
    public function __toString(): string {
        return "ID: {"$this->id}";
    }
}
```

Interface

```
interface Emprutable {
    public function emprunter(string $nom): bool;
    public function retourner(): bool;
    public function getEmprunteur(): ? string;
}
```

Héritage + Interface

```
class Film extends Ressource
    implements Emprutable {
    public function __construct(
        int $id, string $titre
    ) {
        parent::__construct($id, $titre);
    }
    public function getType(): string {
        return "Film";
    }
    public function emprunter(
        string $nom
    ): bool { ... }
}
```

-2pt CHAPITRE 4 : PHP ET PDO

Connexion

```
$pdo = new PDO($dsn, $user, $pass, [
    PDO::ATTR_ERRMODE =>
        PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE =>
        PDO::FETCH_ASSOC,
]);
```

Prepared statements

```
// Param nommes
$stmt = $pdo->prepare(
    "INSERT INTO t (col) VALUES (:col)"
);
$stmt->bindValue(':col', $v, PDO::
    PARAM_STR);
$stmt->execute();
$id = (int) $pdo->lastInsertId();

// Param positionnels
$stmt = $pdo->prepare(
    "SELECT * FROM t WHERE id = ?"
);
$stmt->execute([$id]);
$row = $stmt->fetch(); // une
// ligne
$rows = $stmt->fetchAll(); // toutes
```

-2pt CHAPITRE 5 : SESSIONS & COOKIES

Session

```
session_start(); // avant tout output !
$_SESSION['utilisateur'] = 'admin';
if (!isset($_SESSION['utilisateur'])) {
    header('Location: login.php'); exit
    ;
}
session_unset();
session_destroy(); // deconnexion
```

Cookie

```
// Creer (30 jours)
setcookie('nom', 'val',
    time() + 30*24*3600, '/');

// Lire
echo $_COOKIE['nom'];

// Supprimer
setcookie('nom', '', time()-3600, '/');
```

Password + Redirection

```
$hash = password_hash($mdp,
    PASSWORD_DEFAULT);
if (password_verify($mdp, $hash)) { ...
}
```



```
header('Location: page.php'); exit;
```